

Const-preserving overloads for the strtoux family of functions

Document number: P0282R0
Date: 2016-02-07
Project: Programming Language C++, Library Evolution Working Group
Reply-to: James Touton <bekenn@gmail.com>

Table of Contents

1. [Table of Contents](#)
2. [Overview](#)
3. [Impact On the Standard](#)
4. [Wording](#)

Overview

The C++ standard library adds const-preserving overloads for specific functions from the C library dealing with null-terminated strings. As defined in the C standard, these functions accept `const char*` parameters and return `char*` values derived from those parameters. Calling these functions would effectively result in silent casts from `const char*` to `char*`. To prevent this, C++ replaces these C functions with const-preserving overloads; for each such C library function, the C++ library provides a version accepting `const char*` parameters and a version accepting `char*` parameters. These functions have the same behavior as the C library version, but the return value preserves the type of the argument (`const char*` or `char*`).

This proposal adds the `strtoux` family of functions to the set of C library functions with const-preserving overloads.

The `strtoux` functions convert the initial portion of a string to a numeric value and fill a user-provided "out" parameter with a pointer to the next character in the string. Although the "in" pointer has the type `const char*`, the "out" parameter has the type `char**`, throwing away the `const` qualifier. This is pretty much asking for undefined behavior:

Example (current usage, generates no warnings):

```
const char* PI_STR = "3.14159 or so";  
char* endptr;    // can't use const char*, would fail to compile on the next line  
float pi = strtouf(PI_STR, &endptr); // bad: silent cast to char*  
*endptr = 'x';  // universe explodes
```

Impact On the Standard

The proposal adds a few new overloads covering the C library `strtoux` family of functions and deprecates the existing versions. Apart from the deprecations, the wording is essentially identical to wording for other overloads already present.

Wording

All modifications are presented relative to N4567.

Insert the following new paragraphs in §21.8 [c.strings] after paragraph 13:

The function signature `strtod(const char*, char**)` shall be supplemented with the two declarations:

```
double strtod(const char* nptr, const char** endptr);  
double strtod(char* nptr, char** endptr);
```

both of which shall have the same behavior as the original declaration.

The function signature `strtouf(const char*, char**)` shall be supplemented with the two declarations:

```
float strtouf(const char* nptr, const char** endptr);
```

```
float strtod(      char* nptr,      char** endptr);
```

both of which shall have the same behavior as the original declaration.

The function signature `strtold(const char*, char**)` shall be supplemented with the two declarations:

```
long double strtold(const char* nptr, const char** endptr);
```

```
long double strtold(      char* nptr,      char** endptr);
```

both of which shall have the same behavior as the original declaration.

The function signature `strtol(const char*, char**, int)` shall be supplemented with the two declarations:

```
long int strtol(const char* nptr, const char** endptr, int base);
```

```
long int strtol(      char* nptr,      char** endptr, int base);
```

both of which shall have the same behavior as the original declaration.

The function signature `strtoll(const char*, char**, int)` shall be supplemented with the two declarations:

```
long long int strtoll(const char* nptr, const char** endptr, int base);
```

```
long long int strtoll(      char* nptr,      char** endptr, int base);
```

both of which shall have the same behavior as the original declaration.

The function signature `strtoul(const char*, char**, int)` shall be supplemented with the two declarations:

```
unsigned long int strtoul(const char* nptr, const char** endptr, int base);
```

```
unsigned long int strtoul(      char* nptr,      char** endptr, int base);
```

both of which shall have the same behavior as the original declaration.

The function signature `strtoull(const char*, char**, int)` shall be supplemented with the two declarations:

```
unsigned long long int strtoull(const char* nptr, const char** endptr, int base);
```

```
unsigned long long int strtoull(      char* nptr,      char** endptr, int base);
```

both of which shall have the same behavior as the original declaration.

Add the following new section to Annex D [depr]:

D.7 Null-terminated sequence utilities [depr.c.strings]

The functions from the C standard library shown in Table 160 are deprecated. [*Note*: The corresponding overloads defined in 21.8 are not deprecated. —*end note*]

Table 160 — C string conversion utilities

<u>strtod</u>	<u>strtod</u>	<u>strtold</u>	<u>strtol</u>	<u>strtoll</u>	<u>strtoul</u>	<u>strtoull</u>
---------------	---------------	----------------	---------------	----------------	----------------	-----------------